MLFF STOR1-STOR5 campaign storage-management specification

Status: STOR1 implemented in 0.20.111a0; STOR2 implemented in 0.20.113a0; STOR3 implemented in 0.20.114a0; STOR4 implemented in 0.20.115a0; STOR5 implemented in 0.20.116a0. The storage roadmap is complete.

Purpose

This specification defines storage accounting, checkpoint compaction, automatic safe reclamation, manual tiered cleanup, and optional immutable deduplication/cold archival for MLFF campaigns. Storage policy is subordinate to scientific provenance, restartability, and user-data ownership.

Non-negotiable protections

1. User-supplied source/training/replay/true-label material located in external directories is read-only. Cleanup shall never delete, truncate, rewrite, rename, or move it.
2. A path reference in TOML, SQLite, JSON, a symlink, or an artifact record does not itself confer deletion authority.
3. Destructive operations are limited to campaign-owned artifacts whose resolved locations pass containment/ownership checks.
4. Final selected production models are retained by default.
5. The selected production checkpoint is retained by default.
6. Diagnostic text records, logs, training histories, and compact JSON/CSV evidence are retained by default because they are inexpensive and valuable for provenance.
7. Any artifact required to restart an incomplete stage is protected.
8. Cleanup must be fail-closed under ambiguous ownership or dependency state.

STOR1 - accounting and ownership - implemented in 0.20.111a0

STOR1 is read-only with respect to reclamation. It introduces artifact ownership/retention classification and a storage report with logical bytes, allocated physical bytes when available, unique-inode bytes, largest artifacts, restart/re-evaluation value, and reclamation eligibility.

Deletion eligibility requires campaign ownership, real-path containment, no symlink escape, and a declared retention class. The implemented `storage` CLI emits `results/storage-report.json` and performs no reclamation. It reports path-logical, inode-deduplicated allocated physical, and unique-inode logical bytes, plus artifact families, protected inputs, symlink escapes, largest files/directories, and future reclamation/capability classifications.

The ownership boundary protects the campaign config and all configured source, foundation, replay, and true-label inputs even when physically nested inside the workspace. Existing cleanup/pruning now consumes the same boundary, so external paths referenced by campaign records cannot gain deletion authority. Tests cover malicious/accidental external references, symlink traversal, hardlinks, external materialization roots, and campaign symlinks whose targets must survive. The implementation distinguishes authorization to unlink a contained symlink object from a stricter traversal authorization; cleanup roots are traversed only when their resolved targets remain inside the campaign boundary and outside every protected input.

STOR2 - completed-checkpoint compaction - implemented in 0.20.113a0

Full optimizer checkpoints remain untouched for active/interrupted training and, by default, while the eventual production winner is still unknown. A model-state-only **evaluation state capsule** (the STOR2 evaluation capsules representation) is therefore created only after a run has a durable scientific checkpoint selection: the selected production checkpoint remains byte-for-byte full/restart-capable, while each nonselected checkpoint may be replaced by an authenticated capsule. This slightly more conservative lifetime is intentional: a model-only capsule cannot preserve optimizer continuation for a checkpoint that might later become the production winner.

The v1 capsule preserves the original checkpoint scientific identity and stores the exact MACE model state plus the source checkpoint SHA-256, epoch/run lineage, immutable MACE configuration digest, reconstruction contract, model-state digest, and capsule byte identity. OPT-EVAL1/OPT-EVAL4 source resolution accepts either the original raw checkpoint or a validated capsule, so true-label refresh and later checkpoint re-evaluation can reuse the compact representation without changing selection identity. The completed-run validator likewise accepts the immutable original checkpoint catalog represented by a mixture of raw checkpoints and authenticated capsules.

A raw nonselected checkpoint is removed only after all of the following succeed in order:

- its original SHA-256 and run/epoch lineage are verified;
- the capsule is atomically written and authenticated;
- direct MACE reconstruction from the capsule succeeds;
- reconstructed deployable model state exactly matches reconstruction from the raw checkpoint;
- a real MACE 0.3.16 qualification confirms identical energy, force, and stress outputs;
- the capsule is smaller than the source checkpoint (otherwise the raw checkpoint is retained);
- the capsule record is committed to campaign state and independently read back and re-authenticated; and only then
- the raw checkpoint is deleted through the STOR1 ownership boundary.

Unsupported checkpoint layouts, missing/changed DATA8 configuration, capsule corruption, reconstruction mismatch, ownership ambiguity, or a non-saving capsule all fail closed to ordinary raw-checkpoint retention. The selected production checkpoint and production model remain retained by default. STOR2 introduces no broader cache/artifact reclamation; that authority begins at STOR3.

STOR3 - automatic lifecycle-safe reclamation - implemented in 0.20.114a0

Automatic cleanup is restricted to artifact classes whose removal causes no loss of current scientific result or required restart capability. Examples include garbage, stale staging, obsolete runtime archives after compact diagnostics, orphan payloads, successful-preflight temporaries, and qualified reconstructable graph/view caches.

Checkpoint/capsule removal is permitted only after EVAL-MF lifetimes and final selection are satisfied and a stronger surviving artifact or prediction record preserves the required capability. Automatic low-disk recovery runs this safe policy before interrupting active jobs and never broadens deletion authority merely to meet a free space threshold.

Every automatic cleanup appends one authenticated JSONL event to `results/cleanup-manifest.jsonl`, recording removed paths, bounded pre-deletion filesystem identities, reasons, reclaimed bytes, preserved capabilities, and an explicit empty capability-loss set. The manifest itself is ownership-checked and append-only.

STOR3 also changes low-disk training behavior: the scheduler runs this safe policy before requesting interruption. Active run roots are excluded; only if free space remains below the configured reserve are active children stopped at durable checkpoints. Evaluation graph/view caches become automatically reclaimable only after authoritative evaluation is complete. Prediction caches (`evaluation-predictions`, DATA6/model-sweep, true-label replay) remain outside STOR3 authority because they preserve expensive metric-only/reanalysis capability.

STOR4 - manual tiered reclamation - implemented in 0.20.115a0

Manual cleanup exposes increasingly consequential cumulative tiers through `storage cleanup --tier`. Every invocation first computes and prints/writes a dry-run capability plan. The implemented semantics are ordered:

- `safe`: no scientific/restart capability loss;
- `cache`: only reconstructable acceleration caches; future execution may be slower;

- `recompute`: expensive but reproducible prediction/DATA6-style artifacts; future reselection/re-evaluation may require substantial inference;
- `compact`: nonproduction checkpoints/models and cold materializations after protocol freeze; exact continuation or cheap alternative-model recovery may be lost;
- `archive`: explicitly selected cold reproducibility material converted to verified archival representation.

The implementation distinguishes planning from authorization. `safe` and `cache` retain the pre-STOR4 low-consequence behavior; `recompute` and `compact` are plan-only unless `--apply` is supplied. With STOR5, `storage cleanup --tier archive --apply` first creates and independently verifies a reversible cold archive of every consequential `recompute`/`compact` hot artifact; only after that receipt is committed may those hot representations be removed.

The capability plan states the status of training restart, exact checkpoint re-evaluation, metric-only recomputation, DATA7 reselection, DATA8 rematerialization, current production inference, and verification replay. `recompute` requires authoritative evaluation plus the continued availability of configured reconstruction inputs before scientific prediction/DATA6 caches can be removed. `compact` additionally requires full verification, an authoritative protocol freeze, and a protected production model; it may then remove nonselected evaluation capsules, nonproduction per-run model copies, and hot DATA7/DATA8 materializations. Selected production raw checkpoints, workspace production models, protocol/selection/verification records, and default diagnostics/logs are never STOR4 deletion candidates.

Every consequential manual deletion is recorded in the append-only cleanup manifest with its intentional capability-loss set. STOR3 automatic cleanup continues to require an empty capability-loss set.

STOR5 - immutable deduplication and authenticated cold archival - implemented in 0.20.116a0

STOR5 closes the optional physical-storage layer without changing scientific identities. `storage deduplicate` scans only verified/frozen campaign-owned immutable families. Exact byte duplicates are SHA-256 grouped and, with explicit `--apply`, replaced atomically by same-filesystem hardlinks backed by `.mdstats/content-store/sha256/<prefix>/<digest>`. The command is plan-only by default, excludes active checkpoints/state/logs/production artifacts, refuses to run before verification plus protocol freeze, and never follows symlinks. Storage accounting therefore continues to distinguish logical bytes from inode-deduplicated physical allocation.

Cold archival is exposed through `storage archive create|verify|restore` and through `storage cleanup --tier archive --apply`. Archive candidates are the consequential STOR4 `recompute`/`compact` actions only; safe/cache garbage need not be archived. The archive is a self-contained `tar+gzip` representation under `.mdstats/cold-archive/`, with workspace-relative member paths, per-file SHA-256/size/mode records, an authenticated manifest digest, and an archive SHA-256. Campaign-internal hardlinks are dereferenced when archiving so the archive remains self-contained after hot links and the content store disappear.

Deletion order is binding: plan -> lossless STOR2 checkpoint compaction -> collect the post-STOR2 hot layout -> create archive -> independently read back and authenticate every member -> commit the archive receipt -> delete only the represented hot roots. Archived actions record `archive_restore_available=true` and no irreversible scientific capability loss. Corruption, unsafe paths, missing members, or hash mismatch fail closed before consequential deletion.

`storage archive restore` verifies the registered manifest/archive again, reconstructs files in a campaign-owned staging tree, authenticates staged bytes, preflights all destinations for conflicts, installs only missing exact files, and rehashes the final hot layout. Existing conflicting bytes are never overwritten. A restore receipt is persisted in campaign state/results. `storage archive verify` is read-only.

STOR5 also garbage-collects content-store objects whose last hot hardlink was removed, so prior deduplication cannot defeat later archive/cleanup disk savings. External source, foundation, replay, and true-label paths remain outside both deduplication and archival authority.

Ordering

Storage implementation begins only after EVAL-MF1/MF2 and PREC1-PREC3 define checkpoint, prediction, optimizer-state, precision-stage, and dtype lifetimes. The required order is STOR1 -> STOR2 -> STOR3 -> STOR4 -> STOR5. STOR5 is optional and may not block the higher-priority accounting, checkpoint-compaction, and safe-cleanup work.